

Lab -2 – Task:

Reg.No : AP25110010807

Name : S.Jignesh

Question 1:

Algorithm 1:

Code :

```
main.cpp
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  int main() {
6      int x, n;
7
8      cout << "Enter x: ";
9      cin >> x;
10
11     cout << "Enter n: ";
12     cin >> n;
13
14     int sum = 0;
15
16     if (x == 1) {
17         sum = n + 1;
18     }
19     else {
20         sum = (pow(x, n) - 1) / (x - 1);
21     }
22
23     cout << "Sum = " << sum;
24
25     return 0;
26 }
```

Output:

```
Enter x: 2
Enter n: 3
Sum = 7

...Program finished with exit code 0
Press ENTER to exit console.
```

Algorithm 2:

Code:

```
main.cpp
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  int main() {
6      int x, n;
7      int sum = 0;
8
9      cout << "Enter x: ";
10     cin >> x;
11
12     cout << "Enter n: ";
13     cin >> n;
14
15     for (int i = 0; i < n; i++) {
16         sum = sum + pow(x, i);
17     }
18
19     cout << "Sum = " << sum;
20
21     return 0;
22 }
```

Output:

```
Enter x: 2
Enter n: 3
Sum = 7

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 2:

Algorithm 1:

Code:

```
main.cpp
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  int main() {
6      int n, k;
7
8      cout << "Enter size of array: ";
9      cin >> n;
10
11     int arr[n];
12
13     cout << "Enter elements: ";
14     for (int i = 0; i < n; i++) {
15         cin >> arr[i];
16     }
17
18     cout << "Enter k: ";
19     cin >> k;
20
21     sort(arr, arr + n);
22
23     cout << "Kth smallest element = " << arr[k - 1];
24
25     return 0;
26 }
```

Output:

```
Enter size of array: 5
Enter elements: 1
2
3
4
5
Enter k: 2
Kth smallest element = 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Algorithm 2:

Code:

```
main.cpp
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  int main() {
6      int n, k;
7
8      cout << "Enter size of array: ";
9      cin >> n;
10
11     int arr[n];
12
13     cout << "Enter elements: ";
14     for (int i = 0; i < n; i++) {
15         cin >> arr[i];
16     }
17
18     cout << "Enter k: ";
19     cin >> k;
20
21     priority_queue<int> maxHeap;
22
23     for (int i = 0; i < k; i++) {
24         maxHeap.push(arr[i]);
25     }
26
27     for (int i = k; i < n; i++) {
28
29         if (arr[i] < maxHeap.top()) {
30             maxHeap.pop();
31             maxHeap.push(arr[i]);
32         }
33     }
34
35     cout << "Kth smallest element = " << maxHeap.top();
36
37     return 0;
38 }
```

Output:

```
Enter size of array: 5
Enter elements: 1
2
3
4
5
Enter k: 2
Kth smallest element = 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 3:

Code:

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int data;
6      Node* left;
7      Node* right;
8
9      Node(int value) {
10         data = value;
11         left = NULL;
12         right = NULL;
13     }
14 };
15
16 Node* insert(Node* root, int value) {
17
18     if (root == NULL) {
19         return new Node(value);
20     }
21
22     if (value < root->data) {
23         root->left = insert(root->left, value);
24     }
25     else if (value > root->data) {
26         root->right = insert(root->right, value);
27     }
28
29     return root;
30 }
31
32 Node* findMin(Node* root) {
33
34     if (root->left == NULL) {
35         return root;
36     }
37
38     return findMin(root->left);
39 }
40
```

main.cpp

```
39 }
40
41 Node* deleteNode(Node* root, int value) {
42
43     if (root == NULL) {
44         return root;
45     }
46
47     if (value < root->data) {
48         root->left = deleteNode(root->left, value);
49     }
50     else if (value > root->data) {
51         root->right = deleteNode(root->right, value);
52     }
53     else {
54
55         if (root->left == NULL) {
56             Node* temp = root->right;
57             delete root;
58             return temp;
59         }
60
61         if (root->right == NULL) {
62             Node* temp = root->left;
63             delete root;
64             return temp;
65         }
66
67         Node* temp = findMin(root->right);
68
69         root->data = temp->data;
70
71         root->right = deleteNode(root->right, temp->data);
72     }
73
74     return root;
75 }
76
```

```

main.cpp
76
77 void inorder(Node* root) {
78
79     if (root == NULL) {
80         return;
81     }
82
83     inorder(root->left);
84
85     cout << root->data << " ";
86
87     inorder(root->right);
88 }
89
90 int main() {
91
92     Node* root = NULL;
93
94     root = insert(root, 50);
95     root = insert(root, 30);
96     root = insert(root, 70);
97     root = insert(root, 20);
98     root = insert(root, 40);
99     root = insert(root, 60);
100    root = insert(root, 80);
101
102    cout << "BST after insertion: ";
103    inorder(root);
104
105    root = deleteNode(root, 30);
106
107    cout << "\nBST after deleting 30: ";
108    inorder(root);
109
110    root = deleteNode(root, 70);
111
112    cout << "\nBST after deleting 70: ";
113    inorder(root);
114
115    return 0;
116 }

```

Output:

```

BST after insertion: 20 30 40 50 60 70 80
BST after deleting 30: 20 40 50 60 70 80
BST after deleting 70: 20 40 50 60 80

...Program finished with exit code 0
Press ENTER to exit console.

```